

MODULE 36 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3

The final 25 questions. No topic repeated from papers 1 or 2.

QUESTIONS 25 FORMAT 4 options, A-D MODULE 36 of 36

How to take this paper

- ◆ **Each question is read aloud, then the slide holds** while you decide.
- ◆ **Pause if you need longer.** The hold is a prompt, not a limit.
- ◆ **Answer every question,** even when unsure. There is no penalty for a wrong answer.
- ◆ **The worked answer follows immediately,** with a reason for each wrong option.

ON THE EXAM

The real exam is 45 questions in 90 minutes — two minutes each. Try to keep near that pace.

A BI team's dashboards are queried in short bursts through the morning, with long idle gaps between them. They currently use a classic SQL warehouse, which is stopped overnight to save cost.

Analysts complain that the first query each morning takes several minutes before returning anything.

What should the team change?

- ☐ A Increase the classic warehouse's cluster size
- ☐ B Move to a serverless SQL warehouse, which starts in seconds rather than minutes
- ☐ C Enable Photon on the classic warehouse
- ☐ D Leave the classic warehouse running overnight so it is always warm

◆ PAUSE AND DECIDE

What should the team change?

- A

 Increase the classic warehouse's cluster size
A bigger warehouse takes just as long to start. Size governs query throughput, not startup.
- C

 Enable Photon on the classic warehouse
Photon accelerates query execution. It does nothing to the cold-start delay before the warehouse exists.
- B

 Move to a serverless SQL warehouse, which starts in seconds rather than minutes
Correct. Classic and pro warehouses cold-start in around four minutes; serverless starts in seconds.
- D

 Leave the classic warehouse running overnight so it is always warm
Removes the delay by paying for idle compute all night, which is the cost they were avoiding.

THE SIGNAL

'The first query each morning' is a startup symptom, not a query performance one.

A COPY INTO statement is written with both a FILES clause listing 40 specific files and a PATTERN clause matching a glob. The intention is to load only files matching the pattern from within that list.

The statement is rejected.

Why?

- ☐ A FILES is limited to 40 entries, and the limit was exceeded
- ☐ B PATTERN may only be used with Parquet sources
- ☐ C VALIDATE must be run before a statement using PATTERN
- ☐ D FILES and PATTERN are mutually exclusive; only one may be specified

◆ PAUSE AND DECIDE

Why?

- A

 FILES is limited to 40 entries, and the limit was exceeded
The FILES limit is 1000, and 40 is well inside it.
- B

 PATTERN may only be used with Parquet sources
PATTERN works across the supported formats. It is not format-restricted.
- C

 VALIDATE must be run before a statement using PATTERN
VALIDATE is an optional dry run. It is not a prerequisite.
- D

 FILES and PATTERN are mutually exclusive; only one may be specified
Correct. They are alternative ways of selecting files, not filters that combine.

THE SIGNAL

Two ways to select files, not two filters to intersect. FILES caps at 1000.

An Auto Loader stream reads JSON with schema inference enabled. Downstream, a comparison of `order_total` against a numeric threshold returns no rows, even though large orders plainly exist in the source.

Inspecting the silver table shows `order_total` holds values such as 1499.99 as expected.

What is the most likely cause?

- ☐ A JSON cannot represent decimal values accurately, so comparisons fail
- ☐ B Auto Loader infers JSON and CSV columns as `STRING` by default; the comparison is comparing strings
- ☐ C The threshold must be cast to `STRING` to match the column
- ☐ D The values were rescued into `_rescued_data` and are not really in the column

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 JSON cannot represent decimal values accurately, so comparisons fail
JSON numbers parse fine. The issue is what type Auto Loader assigned on inference.
- B

 Auto Loader infers JSON and CSV columns as STRING by default; the comparison is comparing strings
Correct. `cloudFiles.inferColumnTypes` is false by default. Use schema hints, or enable inference of types.
- C

 The threshold must be cast to STRING to match the column
That would make the comparison lexicographic and still wrong. Fix the column's type instead.
- D

 The values were rescued into `_rescued_data` and are not really in the column
The scenario states the column holds the expected values, so they were not rescued.

THE SIGNAL

The values look right because they are right — as strings. String comparison orders '9' above '1499.99'.

A team is automating the creation of Lakeflow Connect ingestion pipelines through their CI system. Connections for their database sources are created successfully by the pipeline. The connection for a SaaS source that authenticates with browser-based OAuth cannot be created the same way.

Why?

- ☐ A Connections cannot be created programmatically at all; only through Catalog Explorer
- ☐ B The CI principal needs CREATE CONNECTION on the metastore
- ☐ C OAuth U2M connectors require an interactive browser sign-in, so they cannot be created programmatically
- ☐ D SaaS connectors must be created by a metastore admin, and the CI principal is not one

◆ PAUSE AND DECIDE

Why?

- A** Connections cannot be created programmatically at all; only through Catalog Explorer
They can, using the CLI or the API. The exception is browser-based OAuth.
- B** The CI principal needs CREATE CONNECTION on the metastore
A sensible prerequisite, and it would not enable an interactive sign-in either.
- C** OAuth U2M connectors require an interactive browser sign-in, so they cannot be created programmatically
Correct. Somebody must complete the sign-in, which no CI system can do.
- D** SaaS connectors must be created by a metastore admin, and the CI principal is not one
Admin rights would not supply an interactive browser session, which is what is missing.

THE SIGNAL

The same constraint as CI authentication: nobody is present to complete a browser flow.

A bronze table holds a payload column containing JSON strings of varying shape. An analyst needs to read a single field from within the payload for an exploratory query, and does not want to define a schema.

The payload structure changes occasionally, and defining a schema would need maintaining.

What should they use?

- ☐ A from_json with a schema derived by schema_of_json
- ☐ B Cast the payload column to STRUCT
- ☐ C explode on the payload column
- ☐ D The colon operator, which extracts a field from a JSON string without requiring a schema

◆ PAUSE AND DECIDE

What should they use?

- A

`from_json` with a schema derived by `schema_of_json`
Works, and it fixes a schema that then needs maintaining, which the requirement rules out.
- B

 Cast the payload column to `STRUCT`
Casting to `STRUCT` requires stating the struct's fields, which is defining a schema.
- C

`explode` on the payload column
`explode` operates on arrays. The column holds a JSON string.
- D

 The colon operator, which extracts a field from a JSON string without requiring a schema
Correct. It is the lightest option for exploratory access and tolerates shape changes.

THE SIGNAL

'Does not want to define a schema' rules out `from_json` and `STRUCT` alike. `VARIANT` is the durable answer at scale.

A team must ingest from a niche vertical SaaS product used only in their industry. There is no managed connector for it in Lakeflow Connect.

An engineer proposes writing and maintaining a custom Structured Streaming source against the vendor's API.

What should they check first?

- ☐ A Whether a partner connector exists, since partners cover sources Databricks has no managed connector for
- ☐ B Whether Lakehouse Federation can query the SaaS product in place
- ☐ C Nothing; a custom source is the only option when no managed connector exists
- ☐ D Whether the vendor's API supports Auto Loader

◆ PAUSE AND DECIDE

What should they check first?

- A

 Whether a partner connector exists, since partners cover sources Databricks has no managed connector for
Correct. Checking Partner Connect costs minutes and can remove the whole maintenance burden.
- B

 Whether Lakehouse Federation can query the SaaS product in place
Federation covers supported database sources. It is also not an ingestion mechanism.
- C

 Nothing; a custom source is the only option when no managed connector exists
Skips the layer between managed connectors and building your own, which is exactly where partners sit.
- D

 Whether the vendor's API supports Auto Loader
Auto Loader reads files in cloud object storage. An API is the wrong source type for it.

THE SIGNAL

The layer people forget. Between a managed connector and building it yourself sits the partner ecosystem.

A satisfaction_score column is populated only when a customer chooses to leave a rating. About half the rows are null. A dashboard has reported an average of 4.1 for two years.

An engineer adds `fillna(0)` across all numeric columns while tidying the pipeline. The dashboard now shows 2.0.

What happened?

- ☐ A `fillna` overwrote real scores with zero
- ☐ B The dashboard is caching an old value and needs refreshing
- ☐ C `fillna` changed the column type, causing an incorrect calculation
- ☐ D Averages skip nulls, so those rows were previously excluded; filling them with 0 made them count as scores of zero

◆ PAUSE AND DECIDE

What happened?

- A

`fillna` overwrote real scores with zero
`fillna` only replaces nulls. Existing scores are untouched.
- B

 The dashboard is caching an old value and needs refreshing
A cache would show the old number, not a new and lower one.
- C

`fillna` changed the column type, causing an incorrect calculation
Filling an integer column with 0 does not change its type, and a type change would not give a plausible but wrong mean.
- D

 Averages skip nulls, so those rows were previously excluded; filling them with 0 made them count as scores of zero
Correct. A null means 'did not rate'. A zero means 'rated it zero'. Half the data changed meaning.

THE SIGNAL

Filling a null always changes a meaning. Here it changed the meaning of half the data.

A customer table has 52 columns. Fifteen are genuinely optional: a second address line, a middle name, a mobile number and similar. Two are essential: `customer_id` and `email`.

A cleaning step calls `df.dropna()` with no arguments. The silver table ends up with about 4% of the expected rows.

What should the call be?

- ☐ A `df.dropna(subset=['customer_id', 'email'])`
- ☐ B `df.dropna(thresh=52)`
- ☐ C `df.dropna(how='all')`
- ☐ D `df.fillna("")` followed by `df.dropna()`

◆ PAUSE AND DECIDE

What should the call be?

- A

`df.dropna(subset=['customer_id', 'email'])`
Correct. It drops rows missing something essential and ignores nulls in the fifteen optional columns.
- B

`df.dropna(thresh=52)`
Requires all 52 columns to be non-null, which is stricter than the default and keeps even fewer rows.
- C

`df.dropna(how='all')`
Drops only rows where all 52 columns are null, so records missing a customer_id survive.
- D

`df.fillna("")` followed by `df.dropna()`
Keeps every row by turning optional nulls into empty strings, and disguises missing customer_ids as empty strings.

THE SIGNAL

The default is `how='any'` across all 52 columns. With fifteen optional fields, almost every real customer has a null.

A gold table must be fully replaced each night from a silver source whose column set occasionally changes. The team wants the gold table's schema to follow the source automatically.

An engineer currently uses INSERT OVERWRITE and finds that new source columns never appear in gold.

What should be used instead?

- ☐ A TRUNCATE followed by INSERT INTO
- ☐ B CREATE OR REPLACE TABLE, which is DDL and replaces the schema as well as the data
- ☐ C DROP TABLE followed by CREATE TABLE AS SELECT
- ☐ D INSERT OVERWRITE with mergeSchema enabled on the write

◆ PAUSE AND DECIDE

What should be used instead?

- A

 TRUNCATE followed by INSERT INTO
Truncate removes rows and leaves the schema, which is the same limitation as INSERT OVERWRITE.
- B

 CREATE OR REPLACE TABLE, which is DDL and replaces the schema as well as the data
Correct. INSERT OVERWRITE is DML and preserves the existing schema; CREATE OR REPLACE TABLE redefines it.
- C

 DROP TABLE followed by CREATE TABLE AS SELECT
Achieves the same result and discards grants, table history and any downstream references in between.
- D

 INSERT OVERWRITE with mergeSchema enabled on the write
Closer, and the requirement is for the schema to follow the source exactly, including removals, which replacement gives.

THE SIGNAL

DDL replaces the schema. DML keeps it. That single distinction answers the question.

A dashboard shows daily unique devices across 600 million events. It uses `count_distinct` on `device_id` grouped by day, takes fourteen minutes, and times out.

The dashboard exists to spot week-on-week trends. A separate regulatory return also counts unique devices and must be exact.

What should the team do?

- ☐ A Keep `count_distinct` for both and increase the warehouse size
- ☐ B Use `approx_count_distinct` for both, since it is far faster
- ☐ C Use `approx_count_distinct` for the dashboard and keep `count_distinct` for the regulatory return
- ☐ D Use `approx_count_distinct` with `rsd` set to 0.001 for both

◆ PAUSE AND DECIDE

What should the team do?

- A

 Keep count_distinct for both and increase the warehouse size
Buys time at permanent cost, and the exact count still scales badly as events grow.
- C

 Use approx_count_distinct for the dashboard and keep count_distinct for the regulatory return
Correct. Two consumers with different requirements, so two different answers.
- B

 Use approx_count_distinct for both, since it is far faster
Makes the regulatory figure an estimate. 'Must be exact' is a stated requirement.
- D

 Use approx_count_distinct with rsd set to 0.001 for both
Below rsd 0.01 it is more efficient to use count_distinct, so this pays for precision and loses the speed.

THE SIGNAL

Nothing says one query must serve both consumers. The exam rewards noticing when it need not.

A nightly join between a 1.2 TB fact table and a currency_rates lookup has completed in twelve minutes for over a year. In the last fortnight it has slowed to nearly ninety minutes. No code has changed. The Spark UI now shows a large shuffle on both sides where previously there was none.

What is the most likely explanation?

- ☐ A Nulls appeared in the join key, forcing a different strategy
- ☐ B The fact table has grown, so the shuffle takes longer
- ☐ C Adaptive query execution was disabled by a platform change
- ☐ D The lookup grew past the broadcast threshold, so Spark switched to a sort-merge join

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A

 Nulls appeared in the join key, forcing a different strategy
Nulls affect which rows match, not which strategy is chosen. They would change the row count.
- B

 The fact table has grown, so the shuffle takes longer
Gradual growth does not create a shuffle that was not there before. The shuffle is new.
- C

 Adaptive query execution was disabled by a platform change
Possible, and it would not by itself stop a broadcast if the lookup were still small enough at plan time.
- D

 The lookup grew past the broadcast threshold, so Spark switched to a sort-merge join
Correct. Below 10 MB it was broadcast with no shuffle. Once past it, broadcasting stops and both sides shuffle.

THE SIGNAL

'A shuffle where previously there was none' is a strategy change, not a volume change.

A job's first task loads new records and must tell later tasks how many arrived. An expensive rebuild can then be skipped when nothing new came in.

An engineer proposes writing the count into a small Delta table that the downstream task reads.

What is the more idiomatic approach?

- ☐ A Set a task value with `dbutils.jobs.taskValues`, and branch on it with an `If/else` task
- ☐ B A job parameter, set by the first task at runtime
- ☐ C The Delta table approach, since it is the only way to share state between tasks
- ☐ D Combine both tasks into one notebook so the count stays in memory

◆ PAUSE AND DECIDE

What is the more idiomatic approach?

- A

 Set a task value with `dbutils.jobs.taskValues`, and branch on it with an `If/else` task
Correct. Task values exist to return a result into the job environment, and `If/else` conditions test them directly.
- B

 A job parameter, set by the first task at runtime
Job parameters are supplied when the run starts, not written by a task mid-run.
- C

 The Delta table approach, since it is the only way to share state between tasks
It works, and 'only way' is wrong. It also leaves a table whose sole purpose is carrying one number.
- D

 Combine both tasks into one notebook so the count stays in memory
Removes the ability to skip the expensive work as a separate task, which was the requirement.

THE SIGNAL

Tables pass data. Task values pass a decision. A single row count is a decision.

A production job has run nightly for two years. It was created by an engineer who left the company last month, and her account was deprovisioned yesterday.

The job now fails immediately with a permissions error on its source tables. The on-call engineer can query those tables without difficulty.

What is the cause and the fix?

- ☐ A Re-grant SELECT on the source tables to the departed engineer's account
- ☐ B Grant the on-call engineer CAN MANAGE on the job
- ☐ C The job runs as the departed engineer; change the run-as identity to a service principal
- ☐ D The tables were moved when the account was deprovisioned

◆ PAUSE AND DECIDE

What is the cause and the fix?

- A

 Re-grant SELECT on the source tables to the departed engineer's account
Reviving a deprovisioned account to keep production running is the opposite of correct governance.
- B

 Grant the on-call engineer CAN MANAGE on the job
Job privileges are a separate system from run-as. His access changes who can edit the job, not what it can read.
- C

 The job runs as the departed engineer; change the run-as identity to a service principal
Correct. The run-as identity determines data access, and deprovisioning removed the job's ability to read.
- D

 The tables were moved when the account was deprovisioned
Deprovisioning an account does not move tables. It invents a cause when the timing already points at the account.

THE SIGNAL

The on-call engineer's own access is a deliberate red herring. His permissions are unrelated to the job's.

A job's first task is a notebook that reads a job parameter called `batch_date` and works correctly. Its second task is a Python script task that needs the same value.

The script receives nothing and fails on a missing argument. The job parameter is definitely set, as the notebook task proves.

Why, and what is needed?

- ☐ A A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference
- ☐ B Job parameters apply only to the first task in a job
- ☐ C The parameter must be redefined as a task parameter on every task that needs it
- ☐ D The script must call `dbutils.widgets.get`, as the notebook does

◆ PAUSE AND DECIDE

Why, and what is needed?

- A** A Python script task receives an ordered JSON array, so it must reference the job parameter explicitly with a dynamic value reference
Correct. Job parameters are pushed down to key-value tasks. Array tasks must ask for the value by name.
- B** Job parameters apply only to the first task in a job
Invented rule. They are pushed to every task that uses key-value parameters, regardless of position.
- C** The parameter must be redefined as a task parameter on every task that needs it
Would work, and it duplicates the value everywhere, defeating the point of defining it once.
- D** The script must call `dbutils.widgets.get`, as the notebook does
Widgets are the notebook interface for reading a delivered value. They do not deliver anything to a script.

THE SIGNAL

The difference is the task type, not the parameter. Key-value tasks get the push-down; array tasks must ask.

A job is scheduled every twenty minutes. Reviewing the Runs tab, an engineer sees that around a third of runs show a status of Skipped, coloured pink. The runs that do execute succeed normally.

Recent successful runs have been taking between 24 and 31 minutes.

What is happening?

- ☐ A The job is failing intermittently and failures are reported as Skipped
- ☐ B The schedule's time zone is misconfigured
- ☐ C Runs are being skipped because a previous run is still in flight and maximum concurrent runs is 1
- ☐ D The cluster fails to start on those runs

◆ PAUSE AND DECIDE

What is happening?

- A

 The job is failing intermittently and failures are reported as Skipped
Skipped means the run never executed. A failure would be red and would carry an error.
- B

 The schedule's time zone is misconfigured
A time zone offset shifts when runs happen. It does not stop a third of them happening at all.
- C

 Runs are being skipped because a previous run is still in flight and maximum concurrent runs is 1
Correct. Runs take longer than the interval, so triggers land mid-run and the default of 1 causes them to be skipped.
- D

 The cluster fails to start on those runs
That produces a cluster error and a failed run, not a skipped one. The run would have started.

THE SIGNAL

The arithmetic gives it away: a job taking 24–31 minutes cannot run every 20. It has outgrown its schedule.

A team's pipeline deploys to production whenever main is updated. There is no validate step. A pull request renames a variable in databricks.yml but misses one reference elsewhere in the file.

The change is reviewed, approved and merged. The production deployment fails part way, leaving some resources updated and others not.

What would have prevented this?

- ☐ A A more thorough code review
- ☐ B A databricks bundle validate step on the pull request, failing the check before merge
- ☐ C Deploying to production with --force
- ☐ D Running databricks bundle destroy before each deployment

◆ PAUSE AND DECIDE

What would have prevented this?

- A

A more thorough code review

A dangling reference in a long YAML file is what humans skim past and machines catch instantly. Asking people to be careful is not a control.
- B

A databricks bundle validate step on the pull request, failing the check before merge

Correct. Validate resolves the configuration without touching a workspace, so an unresolved reference fails the check.
- C

Deploying to production with --force

Overrides validations, making the pipeline more willing to deploy broken configuration.
- D

Running databricks bundle destroy before each deployment

Tears down production before deploying to it, converting a partial failure into a total outage.

THE SIGNAL

The exam objective names validate before deploy and manage. This is what that ordering is for.

A platform team keeps all code in one repository covering fourteen projects and several thousand files. Engineers report that creating a Git folder takes a very long time, and some Git operations fail outright.

Each engineer works on only one or two of the fourteen projects.

What should they do?

- ☐ A Add the unused projects to .gitignore
- ☐ B Have all engineers share one Git folder to avoid cloning it repeatedly
- ☐ C Increase the Git folder size limit in workspace settings
- ☐ D Use sparse checkout with cone patterns so each engineer clones only the projects they work on

◆ PAUSE AND DECIDE

What should they do?

- A

 Add the unused projects to .gitignore
Prevents new files being tracked. It does not remove committed content or reduce what is cloned.
- B

 Have all engineers share one Git folder to avoid cloning it repeatedly
Solves the cloning cost by creating a worse problem: one working copy, so a colleague switching branch switches it for everybody.
- C

 Increase the Git folder size limit in workspace settings
The limits are platform limits, not a workspace setting you can raise.
- D

 Use sparse checkout with cone patterns so each engineer clones only the projects they work on
Correct. It is the documented answer for monorepos. Splitting the repository is the other legitimate option.

THE SIGNAL

Databricks recommends against monorepo-backed Git folders. Sparse checkout is the mitigation; splitting is the cure.

A platform team enables predictive optimisation across the account. A month later they audit results. Most tables are being optimised and vacuumed automatically.

Three groups receive nothing: tables in a partner-shared external location, their Databricks SQL materialized views, and the streaming tables in their ingestion pipelines.

Why?

- ☐ A Those tables are too small to benefit, so the heuristics skipped them
- ☐ B The account rollout has not yet reached those tables
- ☐ C Predictive optimisation must also be enabled per table
- ☐ D It runs only on Unity Catalog managed tables, and does not run on streaming tables or materialized views

◆ PAUSE AND DECIDE

Why?

- A

 Those tables are too small to benefit, so the heuristics skipped them
The heuristics do skip tables, and that would not correlate so neatly with three distinct object types.
- C

 Predictive optimisation must also be enabled per table
Does not explain why exactly these three categories were affected while everything else worked.
- B

 The account rollout has not yet reached those tables
The rollout is per account, not per table. Everything else in the account is working.
- D

 It runs only on Unity Catalog managed tables, and does not run on streaming tables or materialized views
Correct. Two documented exclusions cover all three groups.

THE SIGNAL

When affected objects share a type rather than a size, look for a categorical rule rather than a heuristic.

For years, a finance team has queried a regulatory table as it stood on any day in the previous quarter, using Delta time travel. Nobody has ever run VACUUM on it.

Predictive optimisation is enabled on the account. Six weeks later an auditor requests the table as at 40 days ago, and the query fails.

What happened, and what should have been done?

- ☐ A OPTIMIZE rewrote the files, which invalidates all previous versions
- ☐ B VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling
- ☐ C Time travel is not supported once predictive optimisation is enabled
- ☐ D The table should have been converted to Iceberg to retain long-range time travel

◆ PAUSE AND DECIDE

What happened, and what should have been done?

- A** OPTIMIZE rewrote the files, which invalidates all previous versions
OPTIMIZE rewrites files and the old ones stay referenced by earlier versions until VACUUM removes them.
- B** VACUUM ran at the default 7-day retention; delta.deletedFileRetentionDuration should have been set before enabling
Correct. Databricks flags this as a warning, because vacuumed files are gone and no setting brings them back.
- C** Time travel is not supported once predictive optimisation is enabled
Time travel still works. What changed is how far back the files exist.
- D** The table should have been converted to Iceberg to retain long-range time travel
PO covers managed Delta and Iceberg alike, with the same VACUUM retention behaviour.

THE SIGNAL

Nobody ever ran VACUUM, so long time travel worked by accident. Enabling PO removed the accident.

A nightly job has begun missing its 07:00 deadline about twice a week. Nobody can say when it started. It has never failed, so no notification has ever fired.

An engineer opens the job's Runs tab and looks at the matrix view for the last month.

What will the matrix view show her most directly?

- ☐ A Nothing useful; she needs the system tables instead
- ☐ B The cost of each run, so she can find the expensive one
- ☐ C Whether the runtime has been increasing, because bar height indicates duration
- ☐ D Which task is failing, indicated in red

◆ PAUSE AND DECIDE

What will the matrix view show her most directly?

- A** Nothing useful; she needs the system tables instead
System tables suit a longer window or many jobs. For one job over one month the matrix answers it immediately.
- B** The cost of each run, so she can find the expensive one
The matrix shows duration and status, not cost. Cost attribution comes from the billing system tables.
- C** Whether the runtime has been increasing, because bar height indicates duration
Correct. A rising staircase across the columns is a visible trend, per task and for the run overall.
- D** Which task is failing, indicated in red
Nothing is failing. The scenario says so, which is why no notification fired.

THE SIGNAL

'It has never failed' rules out looking for red. The information is in the height of the bars, not their colour.

A team has thirty external Delta tables they want to convert to Unity Catalog managed tables, to gain predictive optimisation. They must keep table history, keep existing grants and views working, and minimise downtime for streaming writers.

An engineer proposes CREATE TABLE AS SELECT into new managed tables, then renaming.

What should they use instead?

- ☐ A The CTAS approach as proposed, which is the documented method
- ☐ B ALTER TABLE ... SET MANAGED, which retains history, name, settings, permissions and views
- ☐ C DEEP CLONE into a managed table, then drop the originals
- ☐ D Move the files into the managed storage location manually

◆ PAUSE AND DECIDE

What should they use instead?

- A

 The CTAS approach as proposed, which is the documented method
It works, and it loses table history, creates a new object needing grants reapplied, and requires downtime.
- C

 DEEP CLONE into a managed table, then drop the originals
A real option Databricks names, and SET MANAGED minimises or eliminates downtime compared with it.
- B

 ALTER TABLE ... SET MANAGED, which retains history, name, settings, permissions and views
Correct. Databricks recommends it over CTAS precisely for these reasons, and it minimises downtime.
- D

 Move the files into the managed storage location manually
Unity Catalog manages the storage layout of managed tables. Hand-moving files leaves metastore and storage disagreeing.

THE SIGNAL

The team's three requirements match the three documented benefits of SET MANAGED one for one.

Three weeks after converting a large external table with SET MANAGED, a data science team reports that a Python script has broken. It reads the table by its old storage path rather than by name.

The cluster runs Databricks Runtime 16.4 LTS. Predictive optimisation is enabled.

What happened?

- ☐ A With predictive optimisation on, the old external data is deleted 14 days after conversion; the script needs name-based access
- ☐ B The conversion failed silently and should be re-run
- ☐ C Path-based reads always fail against managed tables from the moment of conversion
- ☐ D The table was rolled back with UNSET MANAGED by someone else

◆ PAUSE AND DECIDE

What happened?

- A** With predictive optimisation on, the old external data is deleted 14 days after conversion; the script needs name-based access
Correct. Three weeks is past 14 days, and path-based redirect requires DBR 18.1 or above.
- B** The conversion failed silently and should be re-run
It plainly succeeded — the table worked by name for three weeks. Only the old path stopped working.
- C** Path-based reads always fail against managed tables from the moment of conversion
They worked for roughly two weeks after conversion, which is why the failure was delayed.
- D** The table was rolled back with UNSET MANAGED by someone else
A rollback deletes the managed data after 7 days, a different symptom on a different timer.

THE SIGNAL

The delay is the clue. Nothing broke at conversion and something broke three weeks later, which points at a timer.

A team must share a curated dataset with an external analyst group. It joins four tables, renames columns, and exposes only aggregated figures. The analysts must not be able to reach any of the four base tables.

A colleague suggests applying row filters and column masks to all four tables instead.

Which approach fits?

- ☐ A A dynamic view over the four tables, with SELECT granted on the view only
- ☐ B Grant the analysts BROWSE on the four tables
- ☐ C Row filters and column masks on the four base tables, then grant SELECT on them
- ☐ D Copy the joined result into a new table on a schedule

◆ PAUSE AND DECIDE

Which approach fits?

- A

 A dynamic view over the four tables, with SELECT granted on the view only
Correct. Views span multiple tables, reshape and aggregate, and users need no access to the base tables.
- B

 Grant the analysts BROWSE on the four tables
BROWSE shows metadata without data. The analysts need the data, in a specific shape.
- C

 Row filters and column masks on the four base tables, then grant SELECT on them
Requires granting SELECT on all four, which the requirement forbids. It also cannot join or aggregate.
- D

 Copy the joined result into a new table on a schedule
Works, and creates a second copy to maintain and secure, plus staleness between refreshes.

THE SIGNAL

'Must not reach the base tables' plus 'joins four tables' is the dynamic view signature.

A catalog holds sixty schemas. A group has been granted SELECT at catalog level, which they need for fifty-nine of them. One schema holds compensation data this group must not see. An engineer proposes DENY SELECT on that one schema so the catalog-level grant can stay.

What should they do instead?

- ☐ A Unity Catalog has no DENY; revoke the catalog grant and grant on the 59 schemas instead
- ☐ B Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema
- ☐ C Move the compensation schema into the hive_metastore catalog so DENY can be used
- ☐ D Use DENY as proposed, which is the standard way to carve out an exception

◆ PAUSE AND DECIDE

What should they do instead?

- A

 Unity Catalog has no DENY; revoke the catalog grant and grant on the 59 schemas instead
Correct. Inheritance flows downward with no way to interrupt it, so the grant must start below the exclusion.
- B

 Grant ALL PRIVILEGES on the catalog and rely on ownership to exclude the schema
Makes the exposure broader, not narrower. Ownership does not exclude anyone from an inherited grant.
- C

 Move the compensation schema into the hive_metastore catalog so DENY can be used
*Moves the most sensitive data into a legacy model
Databricks recommends migrating away from.*
- D

 Use DENY as proposed, which is the standard way to carve out an exception
Standard in many database systems, and Unity Catalog does not support it. The proposal would simply fail.

THE SIGNAL

Unity Catalog is least privilege: ungranted is implicitly denied, so there is nothing to deny.

A governance team is designing a governed tag vocabulary. To make classification searchable, an engineer proposes tags whose values carry an example of the data, such as a sample national identifier from each column.

A colleague objects.

What is the objection?

- ☐ A ABAC policy conditions cannot read tag values, only tag keys
- ☐ B Tags cannot be applied to columns, only to tables and schemas
- ☐ C Tag data can be replicated globally, so tag names and values must never contain personal or sensitive information
- ☐ D Governed tags support keys only, not values

◆ PAUSE AND DECIDE

What is the objection?

- A** ABAC policy conditions cannot read tag values, only tag keys
has_tag_value reads values. It is one of the two functions supported in policy conditions.
- B** Tags cannot be applied to columns, only to tables and schemas
Tags apply to catalogs, schemas, tables, columns, models and volumes.
- C** Tag data can be replicated globally, so tag names and values must never contain personal or sensitive information
Correct. Databricks flags this explicitly. A tag describes the data; it must not contain any of it.
- D** Governed tags support keys only, not values
They are key-value pairs. Values are supported and widely used, such as pii:ssn.

THE SIGNAL

A tag describes the data. It is not the data. pii:ssn is correct; a tag holding an actual identifier is not.

What this paper covered

DOMAIN	QUESTIONS
Domain 1 – Platform	1
Domain 2 – Ingest	5
Domain 3 – Transform	5
Domain 4 – Jobs	4
Domain 5 – CI/CD	2
Domain 6 – Troubleshoot	3
Domain 7 – Govern	5
Total	25

ON THE EXAM

Weighted across all three papers rather than within each one, so a single paper will not match the exam's weights exactly.

Thirty-six modules, seven domains and seventy-five practice questions. **You have covered every objective on the current exam guide.**